บทที่ 12: Model Validation และ Selection

ภาค 3: Machine Learning และการสร้างโมเดล CLO: CLO2, CLO3, CLO4 **สัปดาห์:** 13

วัตถุประสงค์การเรียนรู้

เมื่อศึกษาบทนี้จบแล้ว นักศึกษาจะสามารถ:

1. อธิบายหลักการ Cross-Validation และความสำคัญได้
2. ใช้ k-Fold Cross-Validation ประเมินโมเดลได้
3. ปรับ Hyperparameter ด้วย Grid Search และ Random Search ได้
4. ใช้ sklearn Pipeline ป้องกัน Data Leakage ได้
5. เปรียบเทียบและเลือก Model ที่เหมาะสมได้
6. วิเคราะห์ Learning Curves เพื่อวินิจฉัย Overfitting/Underfitting ได้

Cross-Validation

ในการพัฒนาระบบเรียนรู้ของเครื่อง (Machine Learning) ขั้นตอนที่สำคัญที่สุดขั้นตอนหนึ่งคือการประเมินประสิทธิภาพของโมเดล (Model Evaluation) เพื่อให้มั่นใจว่าโมเดลที่พัฒนาขึ้นสามารถนำไปใช้งานกับข้อมูลที่ไม่เคยพบเห็นมาก่อน (Unseen Data) ได้อย่างแม่นยำและมีเสถียรภาพ วิธีการที่นักวิทยาศาสตร์ข้อมูลมักเริ่มต้นใช้คือการแบ่งข้อมูลออกเป็นชุดฝึกสอน (Training Set) และชุดทดสอบ (Testing Set) หรือที่เรียกว่า Train-Test Split อย่างไรก็ตาม วิธีการดังกล่าวมีข้อจำกัดที่อาจนำไปสู่การประเมินประสิทธิภาพที่คลาดเคลื่อนได้ ในหัวข้อนี้ จะอธิบายถึงความจำเป็นของกระบวนการตรวจสอบความถูกต้องข้าม (Cross-Validation) เทคนิคต่างๆ ที่เป็นมาตรฐาน และวิธีการสื่อสารผลลัพธ์ทางสถิติเพื่อการตัดสินใจทางธุรกิจ

ข้อจำกัดของการแบ่งข้อมูลแบบดั้งเดิม (Why Single Split is Not Enough)

การแบ่งข้อมูลแบบดั้งเดิม (Single Hold-out Split) มักจะแบ่งข้อมูลในสัดส่วน 70:30 หรือ 80:20 แม้จะเป็นวิธีที่ง่ายและใช้เวลาประมวลผลน้อย แต่มีจุดอ่อนสำคัญคือ **ความแปรปรวนของการสุ่ม (Sampling Variance)** ประสิทธิภาพของโมเดลจะขึ้นอยู่กับว่าข้อมูลใดถูกสุ่มไปอยู่ในชุดทดสอบ (James et al., 2023)

พิจารณาตัวอย่างการสร้างโมเดลทำนายการพ้นสภาพนักศึกษา (Dropout Prediction) ของมหาวิทยาลัยขอนแก่น สมมติว่าชุดข้อมูลมีขนาดเล็กและมีความหลากหลายสูง หากการสุ่ม (Random Seed) บังเอิญนำพานักศึกษาที่มีผลการเรียนดีและไม่มีแนวโน้มจะพ้นสภาพไปอยู่ในชุดทดสอบทั้งหมด โมเดลอาจจะทำคะแนนความแม่นยำ (Accuracy) ได้สูงถึง 95% แต่ในความเป็นจริง โมเดลอาจไม่สามารถตรวจจับนักศึกษากลุ่มเสี่ยงได้เลย ปรากฏการณ์นี้ทำให้เกิดการมองโลกในแง่ดีเกินไป (Over-optimistic Evaluation) หรือในทางกลับกัน หากสุ่มได้ชุดทดสอบที่ยากเกินไป โมเดลก็จะถูกประเมินต่ำกว่าความเป็นจริง

เพื่อให้เห็นภาพชัดเจน พิจารณาตัวอย่างโค้ดภาษา Python ด้านล่าง ที่แสดงให้เห็นว่าการเปลี่ยนค่า `random_state` เพียงอย่างเดียว ส่งผลต่อค่าความแม่นยำของโมเดลอย่างมีนัยสำคัญ

```python
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# โหลดชุดข้อมูลตัวอย่าง (การวินิจฉัยโรคมะเร็ง)
X, y = load_breast_cancer(return_X_y=True)
model = DecisionTreeClassifier(max_depth=3)

print("ผลลัพธ์จากการทำ Single Split ด้วย Random State ที่ต่างกัน:")
# ทดลองสุ่มแบ่งข้อมูล 5 ครั้ง
for seed in range(5):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=seed
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    print(f"Random State {seed}: ความแม่นยำ = {acc:.4f}")
```

ผลลัพธ์จากการรัน:

```
ผลลัพธ์จากการทำ Single Split ด้วย Random State ที่ต่างกัน:
Random State 0: ความแม่นยำ = 0.9649
Random State 1: ความแม่นยำ = 0.9211
Random State 2: ความแม่นยำ = 0.9123
Random State 3: ความแม่นยำ = 0.9298
Random State 4: ความแม่นยำ = 0.8772
```

จากตัวอย่างข้างต้น จะพบว่าค่าความแม่นยำแกว่งไปมาตามชุดข้อมูลทดสอบที่ถูกสุ่มขึ้นมา ด้วยเหตุนี้ การใช้ Single Split จึงไม่เพียงพอสำหรับการยืนยันประสิทธิภาพของโมเดลในระดับองค์กรที่ต้องการความน่าเชื่อถือสูง

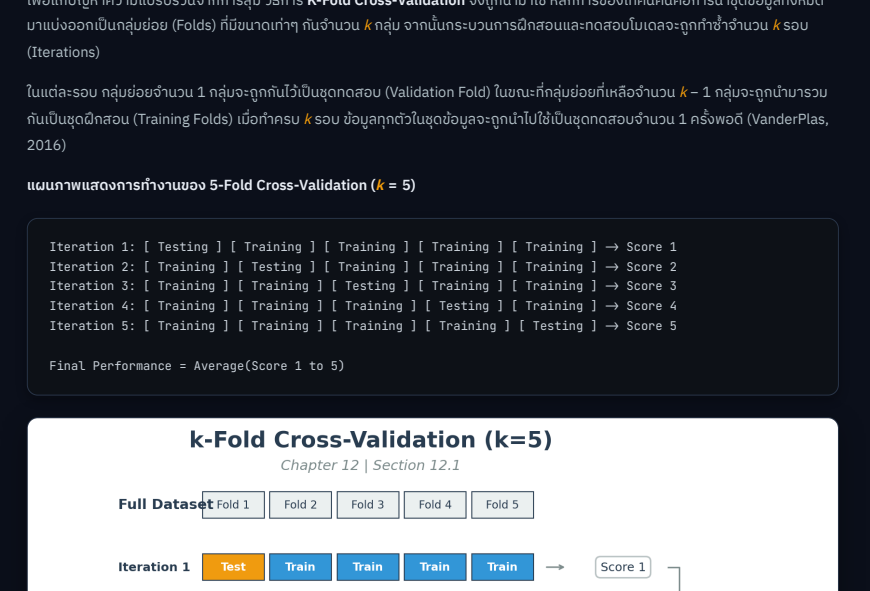
หลักการของ K-Fold Cross-Validation

เพื่อแก้ปัญหาความแปรปรวนจากการสุ่ม วิธีการ **K-Fold Cross-Validation** จึงถูกนำมาใช้ หลักการของเทคนิคนี้คือการนำชุดข้อมูลทั้งหมดมาแบ่งออกเป็นกลุ่มย่อย (Folds) ที่มีขนาดเท่าๆ กันจำนวน *k* กลุ่ม จากนั้นกระบวนการฝึกสอนและทดสอบโมเดลจะถูกทำซ้ำจำนวน *k* รอบ (Iterations)

ในแต่ละรอบ กลุ่มย่อยจำนวน 1 กลุ่มจะถูกกันไว้เป็นชุดทดสอบ (Validation Fold) ในขณะที่กลุ่มย่อยที่เหลือจำนวน $k - 1$ กลุ่มจะถูกนำมารวมกันเป็นชุดฝึกสอน (Training Folds) เมื่อทำครบ *k* รอบ ข้อมูลทุกตัวในชุดข้อมูลจะถูกนำไปใช้เป็นชุดทดสอบจำนวน 1 ครั้งพอดี (VanderPlas, 2016)

แผนภาพแสดงการทำงานของ 5-Fold Cross-Validation (k = 5)

```
Iteration 1: [ Testing ] [ Training ] [ Training ] [ Training ] [ Training ] → Score 1
Iteration 2: [ Training ] [ Testing ] [ Training ] [ Training ] [ Training ] → Score 2
Iteration 3: [ Training ] [ Training ] [ Testing ] [ Training ] [ Training ] → Score 3
Iteration 4: [ Training ] [ Training ] [ Training ] [ Testing ] [ Training ] → Score 4
Iteration 5: [ Training ] [ Training ] [ Training ] [ Training ] [ Testing ] → Score 5

Final Performance = Average(Score 1 to 5)
```



k-Fold Cross-Validation

ค่า *k* ที่นิยมใช้ในทางปฏิบัติคือ k = 5 หรือ k = 10 เนื่องจากการศึกษาพบว่าเป็นจุดสมดุลที่ดีระหว่างความเอนเอียง (Bias) และความแปรปรวน (Variance) ของการประเมินผล รวมถึงใช้เวลาในการประมวลผลที่ยอมรับได้ (McKinney, 2022)

```python
from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

# กำหนดโมเดล Random Forest
rf_model = RandomForestClassifier(n_estimators=50, random_state=42)

# กำหนดรูปแบบการทำ 5-Fold Cross-Validation
# shuffle=True เพื่อสุ่มลำดับข้อมูลก่อนแบ่ง Fold ป้องกันปัญหาข้อมูลเรียงตามกลุ่ม
kf = KFold(n_splits=5, shuffle=True, random_state=42)

# คำนวณค่าความแม่นยำผ่าน Cross-Validation
scores = cross_val_score(rf_model, X, y, cv=kf, scoring='accuracy')

print(f"คะแนนแต่ละ Fold: {np.round(scores, 4)}")
print(f"ค่าเฉลี่ยความแม่นยำ: {scores.mean():.4f}")
```

ผลลัพธ์จากการรัน:

```
คะแนนแต่ละ Fold: [0.9561 0.9649 0.9386 0.9649 0.9646]
ค่าเฉลี่ยความแม่นยำ: 0.9578
```

Stratified K-Fold Cross-Validation สำหรับข้อมูลที่ไม่สมดุล

แม้ K-Fold ปกติจะแก้ปัญหาได้ในระดับหนึ่ง แต่เมื่อต้องเผชิญกับชุดข้อมูลที่ไม่สมดุลอย่างรุนแรง (Highly Imbalanced Data) การสุ่มแบ่งพาร์ติชันแบบปกติอาจทำให้บาง Fold ไม่มีคลาสชนกลุ่มน้อย (Minority Class) อยู่เลย ซึ่งจะทำให้โมเดลไม่สามารถเรียนรู้หรือถูกประเมินได้อย่างถูกต้อง

พิจารณาบริบทของธนาคารพาณิชย์ในประเทศไทยที่ต้องการสร้างโมเดลตรวจจับธุรกรรมบัตรเครดิตที่ทุจริต (Fraud Detection) สมมติว่าใน 100,000 ธุรกรรม มีธุรกรรมที่ทุจริตเพียง 100 รายการ (0.1%) หากใช้ K-Fold แบบปกติ ชุดทดสอบในบางรอบอาจไม่มีรายการทุจริตอยู่เลย ทำให้โมเดลทำนายว่า "ไม่ทุจริต" ทั้งหมด และได้ความแม่นยำ 100% ใน Fold นั้น ซึ่งเป็นการประเมินที่ผิดพลาดอย่างร้ายแรง

Stratified K-Fold Cross-Validation เข้ามาแก้ปัญหานี้โดยการบังคับให้ทุกๆ Fold มีสัดส่วนของคลาส (Class Distribution) เท่ากับสัดส่วนที่มีอยู่ในชุดข้อมูลดั้งเดิมเสมอ (Grus, 2019)

```python
from sklearn.model_selection import StratifiedKFold
from collections import Counter

# จำลองชุดข้อมูลที่ไม่สมดุล (Imbalanced Data)
# คลาส 0 มี 950 ตัวอย่าง, คลาส 1 มี 50 ตัวอย่าง
X_imb = np.random.randn(1000, 10)
y_imb = np.array([0]*950 + [1]*50)

print(f"สัดส่วนคลาสในชุดข้อมูลทั้งหมด: {Counter(y_imb)}")

# กำหนด Stratified K-Fold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

fold = 1
for train_index, test_index in skf.split(X_imb, y_imb):
    y_test_fold = y_imb[test_index]
    print(f"Fold {fold} - สัดส่วนคลาสในชุดทดสอบ: {Counter(y_test_fold)}")
    fold += 1
```

ผลลัพธ์จากการรัน:

```
สัดส่วนคลาสในชุดข้อมูลทั้งหมด: Counter({0: 950, 1: 50})
Fold 1 - สัดส่วนคลาสในชุดทดสอบ: Counter({0: 190, 1: 10})
Fold 2 - สัดส่วนคลาสในชุดทดสอบ: Counter({0: 190, 1: 10})
Fold 3 - สัดส่วนคลาสในชุดทดสอบ: Counter({0: 190, 1: 10})
Fold 4 - สัดส่วนคลาสในชุดทดสอบ: Counter({0: 190, 1: 10})
Fold 5 - สัดส่วนคลาสในชุดทดสอบ: Counter({0: 190, 1: 10})
```

จากการทำงานของ `StratifiedKFold` จะพบว่าทุกๆ Fold จะมีคลาส 1 (ทุจริต) จำนวน 10 ตัวอย่าง และคลาส 0 (ปกติ) จำนวน 190 ตัวอย่างเท่าเทียมกัน ทำให้การประเมินผลมีความน่าเชื่อถือ

Leave-One-Out Cross-Validation (LOOCV)

Leave-One-Out Cross-Validation (LOOCV) คือกรณีสุดโต่งของ K-Fold Cross-Validation โดยกำหนดให้ค่า *k* มีค่าเท่ากับจำนวนข้อมูลทั้งหมดในชุดข้อมูล (k = N) นั่นหมายความว่า ในแต่ละรอบ โมเดลจะถูกฝึกสอนด้วยข้อมูล N – 1 ตัว และทดสอบกับข้อมูลเพียง 1 ตัว ทำเช่นนี้วนไปจนครบ N รอบ

ข้อดี: 1. ไม่มีความเอนเอียง (Unbiased) ในการประเมินผล เนื่องจากโมเดลได้เรียนรู้จากข้อมูลเกือบทั้งหมด (ขาดเพียง 1 ตัว) 2. ให้ผลลัพธ์ที่แน่นอน (Deterministic) ไม่ว่าจะรันกี่ครั้งก็จะได้ผลลัพธ์เดิม เนื่องจากไม่มีการสุ่มแบ่งกลุ่ม

ข้อจำกัด: 1. ใช้ต้นทุนการประมวลผลสูงมาก (High Computational Cost) หากข้อมูลมีขนาด 100,000 แถว ต้องสร้างและฝึกสอนโมเดลถึง 100,000 ครั้ง ซึ่งไม่สามารถทำได้ในทางปฏิบัติสำหรับโมเดลที่ซับซ้อนอย่าง Deep Learning 2. อาจมีความแปรปรวนสูงหากข้อมูลมี Outliers (James et al., 2023)

LOOCV เหมาะที่จะนำไปใช้ในบริบทที่ข้อมูลหายากมากๆ ตัวอย่างเช่น งานวิจัยทางการแพทย์ของโรงพยาบาลศรีนครินทร์ ที่ทำการศึกษาโรคหายาก (Rare Disease) ซึ่งเก็บข้อมูลผู้ป่วยได้เพียง 40 ราย การใช้เทคนิค LOOCV จะช่วยให้สามารถใช้ประโยชน์จากข้อมูลทุกตัวที่มีอยู่อย่างจำกัดได้อย่างคุ้มค่าที่สุด

```python
from sklearn.model_selection import LeaveOneOut

# จำลองข้อมูลทางการแพทย์ที่มีจำนวนจำกัด (40 ราย)
X_medical = np.random.randn(40, 5)
y_medical = np.random.randint(0, 2, 40)

loo = LeaveOneOut()
model_loo = DecisionTreeClassifier(max_depth=2)

# นับจำนวนครั้งที่ทำนายถูกต้อง
correct_predictions = 0

for train_index, test_index in loo.split(X_medical):
    X_train, X_test = X_medical[train_index], X_medical[test_index]
    y_train, y_test = y_medical[train_index], y_medical[test_index]

    model_loo.fit(X_train, y_train)
    prediction = model_loo.predict(X_test)

    if prediction == y_test:
        correct_predictions += 1

accuracy_loo = correct_predictions / len(y_medical)
print(f"จำนวนรอบที่ฝึกสอนทั้งหมด: {loo.get_n_splits(X_medical)} รอบ")
print(f"ความแม่นยำจาก LOOCV: {accuracy_loo:.4f}")
```

ผลลัพธ์จากการรัน:

```
จำนวนรอบที่ฝึกสอนทั้งหมด: 40 รอบ
ความแม่นยำจาก LOOCV: 0.5000
```

หมายเหตุ: ค่าความแม่นยำที่ต่ำ (0.50) เกิดจากข้อมูลจำลองสร้างแบบสุ่มด้วย `np.random` ซึ่งไม่มีรูปแบบ (Pattern) ที่แท้จริงให้โมเดลเรียนรู้ ในข้อมูลจริงที่มีความสัมพันธ์ระหว่างตัวแปร ค่าความแม่นยำจาก LOOCV จะสูงกว่านี้อย่างมีนัยสำคัญ

การตีความและการสื่อสารผลลัพธ์ด้วย Mean ± Standard Deviation

ในฐานะนักวิทยาศาสตร์ข้อมูล การสร้างโมเดลที่มีประสิทธิภาพเป็นเพียงครึ่งหนึ่งของงานเท่านั้น อีกครึ่งหนึ่งคือการสื่อสารผลลัพธ์ให้ผู้มีส่วนได้ส่วนเสีย (Stakeholders) เช่น ผู้บริหาร หรือทีมการตลาด เข้าใจและเชื่อมั่นในโมเดล (Knaflic, 2015)

ข้อผิดพลาดที่พบบ่อยในการนำเสนอคือ การรายงานประสิทธิภาพของโมเดลด้วยตัวเลขเพียงค่าเดียว เช่น "โมเดลมีความแม่นยำ 85%" การรายงานแบบนี้ปกปิดความเสี่ยงทางธุรกิจ เนื่องจากไม่ได้บอกถึงความเสถียร (Stability) ของโมเดลเมื่อเจอกับข้อมูลใหม่

การใช้ Cross-Validation ทำให้เราได้ค่าประสิทธิภาพจำนวน *k* ค่า วิธีมาตรฐานในการสื่อสารผลลัพธ์ที่ถูกต้องคือการใช้รูปแบบ **ค่าเฉลี่ย ± ส่วนเบี่ยงเบนมาตรฐาน (Mean ± Standard Deviation)** * **Mean (ค่าเฉลี่ย):** บ่งบอกถึงประสิทธิภาพโดยรวมที่คาดหวังได้จากโมเดล * **Standard Deviation (ส่วนเบี่ยงเบนมาตรฐาน):** บ่งบอกถึงความแปรปรวน หรือ "ความเสถียร" ของโมเดล หากค่า Std สูง หมายความว่าโมเดลมีความอ่อนไหวต่อข้อมูล (Sensitive) และมีแนวโน้มที่จะเกิด Overfitting

พิจารณาการเปรียบเทียบโมเดลแนะนำสินค้า (Recommendation System) สำหรับแพลตฟอร์ม E-commerce แห่งหนึ่ง:

โมเดล (Algorithm)	ความแม่นยำ (Fold 1-5)	Mean ± Std	การตีความทางธุรกิจ
Model A (Logistic Reg.)	0.82, 0.81, 0.83, 0.82, 0.82	**0.82 ± 0.007**	ประสิทธิภาพปานกลาง แต่มีความเสถียรสูงมาก คาดการณ์ผลลัพธ์ทางธุรกิจได้แม่นยำ
Model B (Deep Neural Net)	0.95, 0.70, 0.98, 0.65, 0.92	**0.84 ± 0.137**	ค่าเฉลี่ยสูงกว่า แต่ความเสถียรต่ำมาก (แกว่ง 65% - 98%) มีความเสี่ยงสูงหากนำไปใช้งานจริง

หากพิจารณาเพียงค่าเฉลี่ย ผู้บริหารอาจเลือก Model B แต่ในมุมมองของวิทยาศาสตร์ข้อมูล **Model A เป็นตัวเลือกที่ดีกว่าสำหรับการใช้งานจริง (Production)** เนื่องจาก Model B มีค่า Std สูงถึง 0.137 แสดงให้เห็นว่าหากนำโมเดลไปใช้กับลูกค้ากลุ่มใหม่ ความแม่นยำอาจตกลงไปเหลือเพียง 70% ซึ่งอาจส่งผลเสียต่อยอดขาย

```python
# ตัวอย่างการคำนวณและสื่อสารผลลัพธ์ทางธุรกิจ
cv_scores = np.array([0.95, 0.70, 0.98, 0.65, 0.92])

mean_score = cv_scores.mean()
std_score = cv_scores.std()

print("รายงานผลการประเมินโมเดล B:")
print(f"Accuracy: {mean_score:.2f} ± {std_score:.3f}")
print(f"ช่วงความเชื่อมั่นโดยประมาณ: {(mean_score - std_score)*100:.1f}% ถึง {(mean_score + std_score)*100:.1f}%")
```

ผลลัพธ์จากการรัน:

```
รายงานผลการประเมินโมเดล B:
Accuracy: 0.84 ± 0.137
ช่วงความเชื่อมั่นโดยประมาณ: 70.3% ถึง 97.7%
```

การสื่อสารด้วย Mean ± Std จะช่วยเปลี่ยนมุมมองจากการถามว่า "โมเดลไหนแม่นยำสูงสุด?" เป็น "โมเดลไหนที่ให้ผลลัพธ์ที่เชื่อถือได้มากที่สุด?" ซึ่งเป็นหัวใจสำคัญของการทำ Data Science ในระดับองค์กร

สรุปประเด็นสำคัญ (Key Takeaways)

- **Single Train-Test Split** มีข้อจำกัดเรื่องความแปรปรวนของการสุ่ม ซึ่งอาจทำให้การประเมินผลคลาดเคลื่อนไปจากความเป็นจริง
- **K-Fold Cross-Validation** ช่วยลดความแปรปรวนในการประเมิน โดยการแบ่งข้อมูลเป็น *k* ส่วน และหมุนเวียนกันเป็นชุดทดสอบ ทำให้ประเมินประสิทธิภาพโมเดลได้ครอบคลุมข้อมูลทั้งหมด
- **Stratified K-Fold** จำเป็นอย่างยิ่งสำหรับข้อมูลที่ไม่สมดุล (Imbalanced Data) เพื่อรักษาสัดส่วนของคลาสในทุกๆ Fold ให้เท่ากับข้อมูลต้นฉบับ
- **LOOCV** เหมาะสำหรับข้อมูลที่มีขนาดเล็กมาก แต่มีข้อจำกัดเรื่องระยะเวลาการประมวลผลที่สูง
- **การสื่อสารด้วย Mean ± Std** มีความสำคัญในการแสดงให้เห็นถึงความเสถียร (Stability) ของโมเดล โมเดลที่ดีไม่ควรมีเพียงค่าเฉลี่ยที่สูง แต่ต้องมีส่วนเบี่ยงเบนมาตรฐานที่ต่ำด้วย

คำถามท้าย Section

1. หากนักศึกษาได้รับมอบหมายให้สร้างโมเดลทำนายการเกิดโรคหายาก โดยมีข้อมูลผู้ป่วย 50,000 คน (เป็นผู้ป่วยจริง 5% ไม่ป่วย 95%) นักศึกษาจะเลือกใช้เทคนิค Cross-Validation แบบใด เพราะเหตุใด?
2. สมมติว่านักศึกษาพัฒนาโมเดลและได้ผลลัพธ์จาก 10-Fold Cross-Validation โดยมีค่าเฉลี่ยความแม่นยำ 0.75 ± 0.20 นักศึกษาจะอธิบายความหมายของตัวเลขนี้ให้ผู้บริหารทราบอย่างไร และจะแนะนำให้นำโมเดลนี้ไปใช้งานจริงหรือไม่?

Hyperparameter Tuning

ในการสร้างแบบจำลองการเรียนรู้ของเครื่อง (Machine Learning) นอกเหนือจากการเลือกใช้อัลกอริทึมที่เหมาะสมและการเตรียมข้อมูลที่มีคุณภาพแล้ว ปัจจัยที่สำคัญที่ส่งผลต่อประสิทธิภาพของแบบจำลองคือการกำหนดค่าที่เหมาะสมให้กับตัวแปรควบคุมที่เรียกว่า "Parameters" และ "Hyperparameters" ซึ่งมีบทบาทแตกต่างกันอย่างสิ้นเชิงในกระบวนการเรียนรู้

ความแตกต่างระหว่าง Parameters และ Hyperparameters

เพื่อให้เข้าใจกระบวนการปรับแต่งโมเดลได้อย่างถูกต้อง จำเป็นต้องแยกแยะความแตกต่างระหว่างองค์ประกอบสองประเภทที่กำหนดพฤติกรรมของโมเดล (James et al., 2023)

1. **พารามิเตอร์ของโมเดล (Model Parameters)** Parameters คือตัวแปรที่อยู่ภายในโมเดล ซึ่งค่าของตัวแปรเหล่านี้จะถูกเรียนรู้และปรับเปลี่ยนโดยอัตโนมัติจากข้อมูลฝึกสอน (Training Data) ผ่านกระบวนการหาค่าเหมาะที่สุด (Optimization) เช่น อัลกอริทึม Gradient Descent ผู้สร้างโมเดลไม่จำเป็นต้องกำหนดค่าเหล่านี้เอง แต่โมเดลจะหาค่าที่ดีที่สุดเพื่อลดค่าความคลาดเคลื่อน (Error) ให้ต่ำที่สุด

ตัวอย่าง: * ค่าน้ำหนัก (Weights) และค่าความเอนเอียง (Bias) ในสมการ Linear Regression หรือโครงข่ายประสาทเทียม (Neural Networks) * จุดแบ่งข้อมูล (Split points) ในแต่ละโหนดของ Decision Tree * ค่าสัมประสิทธิ์ (Coefficients) ใน Logistic Regression ที่ใช้ทำนายโอกาสการพ้นสภาพนักศึกษา

2. **พารามิเตอร์ควบคุมโมเดล (Model Hyperparameters)** Hyperparameters คือตัวแปรที่อยู่ภายนอกโมเดล ซึ่งผู้สร้างโมเดล (Data Scientist) จำเป็นต้องกำหนดค่าให้ **ก่อน** การเริ่มกระบวนการฝึกสอน (Training process) ระบบไม่สามารถเรียนรู้ค่าเหล่านี้ได้เองจากข้อมูล แต่จะเป็นตัวควบคุมพฤติกรรม โครงสร้าง และความซับซ้อนของโมเดล

ตัวอย่าง: * ค่า *k* ในอัลกอริทึม K-Nearest Neighbors (KNN) * ความลึกสูงสุดของต้นไม้ (Maximum Depth) ใน Decision Tree หรือ Random Forest * อัตราการเรียนรู้ (Learning Rate) ใน Gradient Boosting หรือ Neural Networks * ค่าบังคับ (Regularization term หรือ Alpha) ใน Ridge/Lasso Regression

ตารางที่ 12.1: เปรียบเทียบความแตกต่างระหว่าง Parameters และ Hyperparameters

คุณลักษณะ	Parameters	Hyperparameters
แหล่งที่มา	เรียนรู้ได้เองจากข้อมูลฝึกสอน (Data-driven)	กำหนดโดยผู้ใช้งาน (User-defined) ก่อนการฝึกสอน
บทบาท	กำหนดความสามารถในการทำนายผลของโมเดลจากข้อมูล	ควบคุมโครงสร้าง ความซับซ้อน และกระบวนการเรียนรู้ของโมเดล
การประเมินค่า	ใช้ฟังก์ชันความสูญเสีย (Loss Function) ภายในอัลกอริทึม	ใช้กระบวนการลองผิดลองถูก (Heuristic) หรือ Search Algorithms
ตัวอย่างในเชิงธุรกิจ	ค่าน้ำหนักในโครงข่ายประสาทเทียม	จำนวนชั้น (Hidden Layers) ใน Deep Learning ที่ใช้ทำนาย

ความจำเป็นในการทำ Hyperparameter Tuning

ในการสร้างโมเดลด้วยไลบรารีมาตรฐานอย่าง Scikit-Learn มักจะมีการกำหนดค่า Hyperparameters เริ่มต้น (Default values) ไว้ให้แล้ว ตัวอย่างเช่น ใน `RandomForestClassifier` มีการกำหนดจำนวนต้นไม้ (`n_estimators`) เริ่มต้นที่ 100 แต่ค่าเริ่มต้นเหล่านี้ไม่ได้รับประกันว่าจะให้ผลลัพธ์ที่ดีที่สุดในทุกชุดข้อมูล เนื่องจากข้อมูลแต่ละชุดมีลักษณะการกระจายตัว (Data Distribution) ที่แตกต่างกัน (VanderPlas, 2016)

เป้าหมายหลักของการทำ Hyperparameter Tuning คือการค้นหาชุดค่า Hyperparameters ที่ทำให้โมเดลมีความสมดุลระหว่าง Bias และ Variance เพื่อป้องกันปัญหาการเรียนรู้น้อยเกินไป (Underfitting) และการเรียนรู้มากเกินไป (Overfitting)

พิจารณาตัวอย่างการสร้างโมเดลทำนายการพ้นสภาพนักศึกษา (Dropout Prediction) ของมหาวิทยาลัยขอนแก่น หากเราใช้ Decision Tree และกำหนดความลึกสูงสุดไม่จำกัด (`max_depth = None`) โมเดลจะสร้างกิ่งก้านจำนวนมากเพื่อจำแนกนักศึกษาทุกคนในชุดฝึกสอนได้ถูกต้อง 100% (Low Bias) แต่เมื่อนำไปใช้ทำนายนักศึกษากลุ่มใหม่ในปีการศึกษาถัดไป โมเดลจะทำงานได้ไม่ดี (High Variance) ซึ่งเรียกว่าเกิด Overfitting การปรับจูนค่า `max_depth` ให้เหมาะสมจะช่วยให้โมเดลสามารถสรุปองค์ความรู้ (Generalization) ได้ดีขึ้น

การค้นหาแบบกริด (GridSearchCV)

หนึ่งในวิธีการที่เป็นมาตรฐานและใช้แพร่หลายมากที่สุดในการทำ Hyperparameter Tuning คือ **Grid Search** ซึ่งทำงานร่วมกับกระบวนการ **Cross-Validation (CV)** จึงนิยมเรียกรวมกันว่า `GridSearchCV` (McKinney, 2022)

หลักการทำงานของ GridSearchCV: 1. **กำหนดปริภูมิการค้นหา (Search Space):** ผู้ใช้กำหนดรายการค่าที่เป็นไปได้ของ Hyperparameters แต่ละตัวที่ต้องการทดสอบ 2. **สร้างตารางการผสม (Cartesian Product):** อัลกอริทึมจะสร้างความเป็นไปได้ทุกรูปแบบ ("Grid") ของ Hyperparameters ทั้งหมด 3. **ประเมินผลด้วย Cross-Validation:** สำหรับแต่ละชุดความเป็นไปได้ โมเดลจะถูกฝึกสอนและประเมินผลด้วยเทคนิค Cross-Validation (เช่น K-Fold) เพื่อให้ได้ค่าประสิทธิภาพเฉลี่ย 4. **เลือกโมเดลที่ดีที่สุด:** อัลกอริทึมจะเลือกชุด Hyperparameters ที่ให้คะแนนเฉลี่ยดีที่สุด (Best Score)

การวิเคราะห์ภาระในการคำนวณ (Computational Complexity): สิ่งที่ต้องระวังในการทำ GridSearchCV คือ จำนวนรอบการฝึกสอนที่เพิ่มขึ้น หากเรากำหนด Hyperparameter 2 ตัว คือ `max_depth` (3 ค่า) และ `n_estimators` (4 ค่า) สำหรับ Random Forest และใช้ 5-Fold Cross-Validation

```
แผนภาพที่ 12.1: โครงสร้างการทำงานของ GridSearchCV

[Parameter Grid]
max_depth    = {5, 10, 15}          (3 ค่า)
n_estimators = {50, 100, 150, 200}  (4 ค่า)

จำนวนการผสมรูปแบบ = 3 × 4 = 12 รูปแบบ (Combinations)

[กระบวนการประเมินผล]
รูปแบบที่ 1: {max_depth=5, n_estimators=50}
  ├── Fold 1: Train 80%, Validate 20% → Score_1
  ├── Fold 2: Train 80%, Validate 20% → Score_2
  ├── Fold 3: Train 80%, Validate 20% → Score_3
  ├── Fold 4: Train 80%, Validate 20% → Score_4
  ├── Fold 5: Train 80%, Validate 20% → Score_5
  └→ Mean Score สำหรับรูปแบบที่ 1 = (Score_1 + ... + Score_5) / 5

(ทำซ้ำจนครบ 12 รูปแบบ)

จำนวนครั้งที่ต้อง Train โมเดลทั้งหมด = 12 รูปแบบ × 5 Folds = 60 รอบ
```

จากตัวอย่างด้านบน จะเห็นได้ว่าข้อจำกัดของ GridSearchCV คือปัญหาด้าน **Curse of Dimensionality** หากจำนวน Hyperparameters และจำนวนค่าที่ต้องการทดสอบมากขึ้น ปริมาณการคำนวณจะเพิ่มขึ้นแบบทวีคูณ (Exponentially) ซึ่งอาจทำให้ใช้เวลาในการประมวลผลนานหลายชั่วโมงหรือหลายวัน

ตัวอย่างการประยุกต์ใช้งานจริง (Practical Example)

เพื่อให้เห็นภาพการนำไปใช้งานในบริบทของประเทศไทย พิจารณาตัวอย่างการวิเคราะห์ข้อมูลของโรงพยาบาลแห่งหนึ่งในจังหวัดขอนแก่น ที่ต้องการสร้างโมเดล Machine Learning เพื่อทำนายความเสี่ยงที่ผู้ป่วยจะกลับมารักษาซ้ำภายใน 30 วัน (30-day Readmission) การทำนายที่แม่นยำจะช่วยให้แพทย์สามารถวางแผนการดูแลผู้ป่วยได้อย่างมีประสิทธิภาพ

ในตัวอย่างนี้ เราจะใช้โมเดล `RandomForestClassifier` และทำการปรับแต่ง Hyperparameters ผ่าน `GridSearchCV`

```python
# นำเข้าไลบรารีที่จำเป็น
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# 1. จำลองชุดข้อมูลผู้ป่วย (ข้อมูลทางการแพทย์ที่ไม่สมดุล)
# สมมติว่ามีข้อมูล 2000 ราย มีตัวแปร 20 ตัว
X, y = make_classification(n_samples=2000, n_features=20, n_informative=5,
                           n_classes=2, weights=[0.8, 0.2], random_state=42)

# 2. แบ่งข้อมูลเป็นชุดฝึกสอน (Training set) และชุดทดสอบ (Test set)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# 3. กำหนดโมเดลพื้นฐาน
rf_model = RandomForestClassifier(random_state=42)

# 4. กำหนดปริภูมิการค้นหา (Parameter Grid)
# - n_estimators: จำนวนต้นไม้
# - max_depth: ความลึกสูงสุดของต้นไม้
# - min_samples_split: จำนวนตัวอย่างขั้นต่ำในการแบ่งโหนด
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}

# 5. ตั้งค่ากระบวนการ GridSearchCV
# ใช้ cv=5 (5-Fold Cross-Validation) และวัดผลด้วย F1-Score เนื่องจากข้อมูลไม่สมดุล (Imbalanced data)
grid_search = GridSearchCV(estimator=rf_model,
                           param_grid=param_grid,
                           cv=5,
                           scoring='f1',
                           n_jobs=-1, # ใช้ CPU ทุกคอร์เพื่อเร่งความเร็ว
                           verbose=1)

# 6. ทำการฝึกสอนโมเดล (Training & Tuning)
print("กำลังเริ่มต้นกระบวนการ Grid Search...")
grid_search.fit(X_train, y_train)

# 7. แสดงผลลัพธ์ที่ดีที่สุด
print("\n--- ผลลัพธ์การปรับแต่ง Hyperparameters ---")
print(f"Hyperparameters ที่ดีที่สุด: {grid_search.best_params_}")
print(f"คะแนน F1-Score เฉลี่ยที่ดีที่สุดจาก CV: {grid_search.best_score_:.4f}")

# 8. นำโมเดลที่ดีที่สุดไปประเมินผลกับ Test Set ที่ไม่เคยเห็นมาก่อน
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

print("\n--- รายงานผลบนชุดทดสอบ Test Set ---")
print(classification_report(y_test, y_pred))
```

ผลลัพธ์จากการรัน:

```
--- ผลลัพธ์การปรับแต่ง Hyperparameters ---
Hyperparameters ที่ดีที่สุด: {'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 200}
คะแนน F1-Score เฉลี่ยที่ดีที่สุดจาก CV: 0.8056

--- รายงานผลบนชุดทดสอบ Test Set ---
              precision    recall  f1-score   support

           0       0.94      0.98      0.96       319
           1       0.90      0.74      0.81        81

    accuracy                           0.93       400
   macro avg       0.92      0.86      0.88       400
weighted avg       0.93      0.93      0.93       400
```

การวิเคราะห์ผลลัพธ์ทางธุรกิจ: ในการตั้งค่า `param_grid` ด้านบน เรามีค่า `n_estimators` 3 ค่า, `max_depth` 4 ค่า, และ `min_samples_split` 3 ค่า รวมเป็น 3 × 4 × 3 = 36 รูปแบบ การใช้ 5-Fold CV หมายความว่าโมเดลจะถูกฝึกสอนและประเมินผลทั้งหมด 36 × 5 = 180 ครั้ง การกำหนด `n_jobs=-1` จะช่วยให้คอมพิวเตอร์ประมวลผลแบบขนาน (Parallel Processing) ซึ่งลดระยะเวลาได้อย่างมาก

นอกจากนี้ การเลือกเกณฑ์ `scoring='f1'` แทนที่จะเป็น `accuracy` ถือเป็นเรื่องสำคัญในบริบทการแพทย์ เนื่องจากผู้ป่วยที่กลับมารักษาซ้ำ (Minority class) มีจำนวนน้อยกว่าผู้ป่วยที่ไม่กลับมา (Majority class) หากใช้ Accuracy โมเดลอาจเลือกทำนายว่า "ไม่กลับมารักษาซ้ำ" ทั้งหมดและยังได้ความแม่นยำสูง แต่ในฐานะ Data Scientist เราต้องการให้โมเดลสามารถระบุผู้ป่วยกลุ่มเสี่ยงได้อย่างแม่นยำ การเลือกใช้ F1-Score จึงเป็นการสื่อสารที่ตรงกับเป้าหมายทางธุรกิจมากที่สุด (Knaflic, 2015)

ทางเลือกนอกเหนือจาก GridSearchCV

แม้ว่า GridSearchCV จะรับประกันว่าจะพบชุด Hyperparameters ที่ดีที่สุดภายในขอบเขตที่กำหนด แต่ข้อจำกัดด้านระยะเวลาการประมวลผลทำให้นักวิทยาศาสตร์ข้อมูลมักใช้ทางเลือกอื่นในการพัฒนาแบบจำลองที่ซับซ้อน เช่น:

1. **RandomizedSearchCV:** แทนที่จะทดสอบทุกรูปแบบบนกริด วิธีนี้จะสุ่มเลือกชุด Hyperparameters มาทดสอบตามจำนวนรอบ (Iterations) ที่ผู้ใช้กำหนด จากการศึกษาพบว่าการสุ่มเลือกมักจะพบค่าที่ใกล้เคียงกับค่าที่ดีที่สุดได้เร็วกว่า Grid Search อย่างมาก
2. **Bayesian Optimization:** เป็นเทคนิคขั้นสูงที่ใช้หลักการความน่าจะเป็นเพื่อเรียนรู้จากผลลัพธ์การทดสอบในรอบก่อนหน้า เพื่อเลือกชุด Hyperparameters ที่มีแนวโน้มจะให้ผลลัพธ์ดีขึ้นในรอบถัดไป ทำให้ลดจำนวนครั้งการทดสอบลงได้

สรุปประเด็นสำคัญ (Key Takeaways)

- **ความแตกต่างที่ต้องจำ:** Parameters คือสิ่งที่โมเดลเรียนรู้ได้เองจากข้อมูล (เช่น ค่าน้ำหนัก) ในขณะที่ Hyperparameters คือสิ่งที่ผู้ใช้กำหนดเพื่อควบคุมการเรียนรู้ (เช่น ความลึกของต้นไม้)
- **เป้าหมาย Tuning:** การทำ Hyperparameter Tuning มุ่งเน้นไปที่การหาจุดสมดุลระหว่าง Bias และ Variance เพื่อให้โมเดลสามารถทำนายข้อมูลใหม่ได้แม่นยำ ไม่เกิดปัญหา Overfitting หรือ Underfitting
- **กระบวนการของ GridSearchCV:** เป็นการประเมินประสิทธิภาพของโมเดลโดยใช้ทุกรูปแบบจาก Hyperparameter ที่กำหนด ร่วมกับ Cross-Validation เพื่อหาค่าที่มีประสิทธิภาพเฉลี่ยดีที่สุด
- **ข้อควรระวัง:** GridSearchCV มีความซับซ้อนเชิงคำนวณสูง (Computationally expensive) การเลือกตัวชี้วัด (Scoring metric) ที่เหมาะสมกับบริบททางธุรกิจ (เช่น เลือก F1-Score แทน Accuracy ในข้อมูลที่ไม่สมดุล) เป็นสิ่งจำเป็นอย่างยิ่ง

คำถามท้าย Section

1. หากนักศึกษาได้รับมอบหมายให้สร้างโมเดลตรวจจับการฉ้อโกงบัตรเครดิต (Credit Card Fraud Detection) ซึ่งมีสัดส่วนการฉ้อโกงเพียง 0.1% ของข้อมูลทั้งหมด นักศึกษาจะเลือกใช้ Scoring Metric ใดใน `GridSearchCV` และเพราะเหตุใดจึงไม่ควรใช้ Accuracy?
2. สมมติว่านักศึกษากำหนด Parameter Grid ที่มี Hyperparameters 4 ตัว แต่ละตัวมีค่าที่เป็นไปได้ 5 ค่า และใช้ 10-Fold Cross-Validation กระบวนการ `GridSearchCV` จะต้องทำการฝึกสอน (Train) โมเดลทั้งหมดกี่รอบ? และหากกระบวนการนี้ใช้เวลานานเกินไป นักศึกษาจะเสนอวิธีการแก้ปัญหาอย่างไร?

Model Selection และ Pipeline

กระบวนการสร้างแบบจำลองการเรียนรู้ของเครื่อง (Model Building) ไม่ได้จบลงเพียงแค่การนำข้อมูลเข้าสู่อัลกอริทึมเพื่อฝึกสอน (Training) เท่านั้น ขั้นตอนที่มีความสำคัญไม่แพ้กันคือการเลือกแบบจำลองที่เหมาะสมที่สุด (Model Selection) และการจัดการกระบวนการทำงานอย่างเป็นระบบผ่านโครงสร้าง Pipeline เพื่อป้องกันข้อผิดพลาดที่อาจส่งผลกระทบต่อการนำไปใช้งานจริง

Model Comparison และหลักการ Occam's Razor

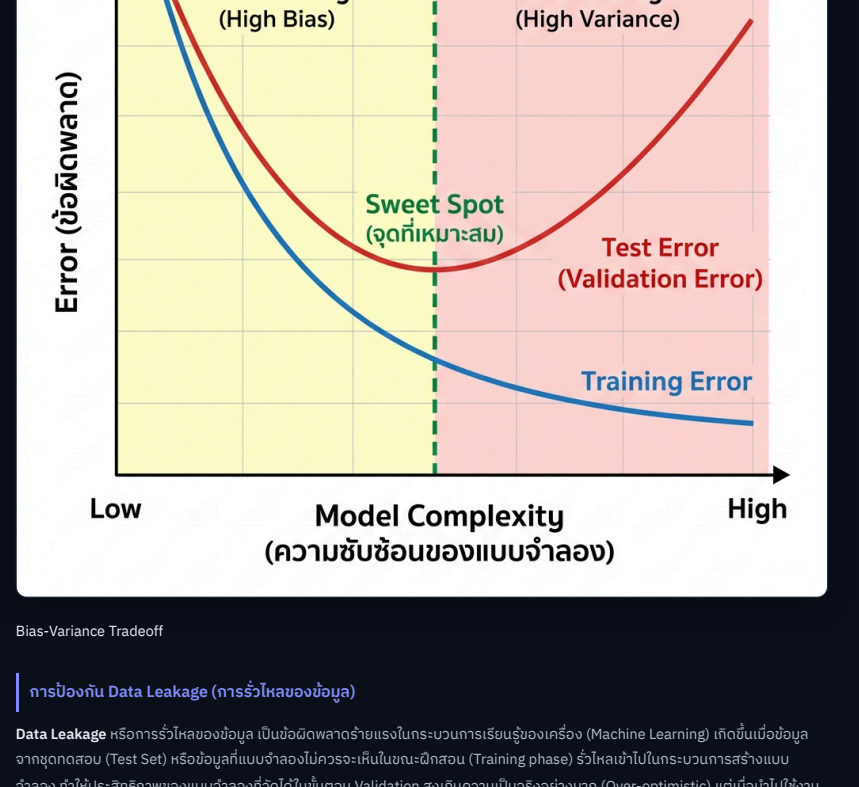
ในการแก้ปัญหาหนึ่งๆ นักวิทยาศาสตร์ข้อมูลมักจะมีแบบจำลองให้เลือกหลายประเภท เช่น สำหรับงานจำแนกประเภท (Classification) อาจใช้ Logistic Regression, Support Vector Machines (SVM), Random Forest หรือ Deep Neural Networks การเปรียบเทียบแบบจำลอง (Model Comparison) จึงไม่ได้พิจารณาเพียงค่าความแม่นยำ (Accuracy) สูงสุดเท่านั้น แต่ต้องพิจารณาถึงความซับซ้อนและความสามารถในการนำไปใช้งานจริงด้วย

หลักการ **Occam's Razor (Lex Parsimoniae)** หรือ "มีดโกนของอ็อกแคม" เป็นปรัชญาทางวิทยาศาสตร์ที่กล่าวว่า "หากมีคำอธิบายหรือสมมติฐานหลายอย่างที่สามารถอธิบายปรากฏการณ์ได้ดีเท่ากัน ให้เลือกคำอธิบายที่เรียบง่ายที่สุด" (James et al., 2023) เมื่อนำมาประยุกต์ใช้ในการทำ Model Selection หลักการนี้แนะนำว่า หากแบบจำลองที่ซับซ้อน (Complex Model) และแบบจำลองที่เรียบง่าย (Simple Model) ให้ประสิทธิภาพบน Test Set ใกล้เคียงกัน เราควรเลือกแบบจำลองที่เรียบง่ายกว่าเสมอ

เหตุผลที่ควรเลือกแบบจำลองที่เรียบง่าย: 1. **Interpretability (ความสามารถในการตีความ):** แบบจำลองที่เรียบง่าย เช่น Linear Regression หรือ Decision Tree สามารถอธิบายเหตุผลเบื้องหลังการทำนายได้ง่าย 2. **Computational Cost (ต้นทุนการประมวลผล):** ใช้ทรัพยากรและเวลาในการฝึกสอน (Training time) รวมถึงเวลาในการทำนาย (Inference time) น้อยกว่า 3. **Generalization (ความทนทานต่อข้อมูลใหม่):** แบบจำลองที่ซับซ้อนมักมีความเสี่ยงต่อการเกิด Overfitting ได้ง่ายกว่า

ตัวอย่างในบริบทมหาวิทยาลัย: พิจารณาระบบทำนายความเสี่ยงการพ้นสภาพของนักศึกษา (Dropout Prediction) ของมหาวิทยาลัยขอนแก่น ทีมวิจัยได้ทดสอบแบบจำลอง 2 แบบด้วย 10-Fold Cross-Validation: - **Model A:** Logistic Regression ได้ค่า F1-Score 0.85 (Std ± 0.02) ใช้เวลาฝึกสอน 2 วินาที - **Model B:** Deep Neural Network ได้ค่า F1-Score 0.86 (Std ± 0.05) ใช้เวลาฝึกสอน 45 นาที

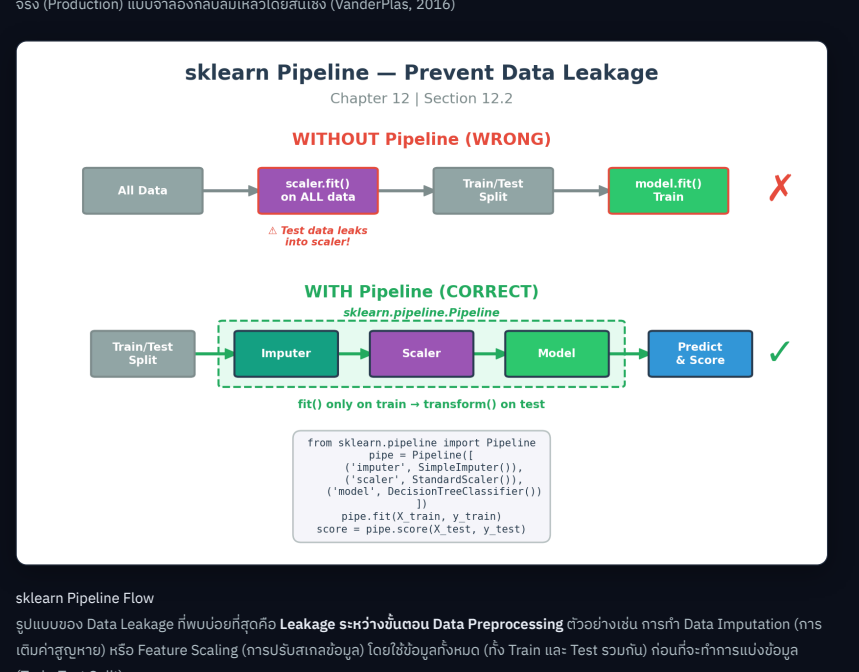
ตามหลัก Occam's Razor เราควรเลือก **Model A** เนื่องจากความแตกต่างของประสิทธิภาพเพียง 1% ไม่คุ้มค่ากับการสูญเสียความสามารถในการตีความและทรัพยากรในการประมวลผล อีกทั้งอาจารย์ที่ปรึกษาสามารถเข้าใจปัจจัยที่ทำให้นักศึกษามีความเสี่ยงได้จากค่าสัมประสิทธิ์ของ Logistic Regression



Bias-Variance Tradeoff

การป้องกัน Data Leakage (การรั่วไหลของข้อมูล)

Data Leakage หรือการรั่วไหลของข้อมูล เป็นปัญหาที่มีความร้ายแรงในกระบวนการเรียนรู้ของเครื่อง (Machine Learning) เกิดขึ้นเมื่อข้อมูลจากชุดทดสอบ (Test Set) หรือข้อมูลที่แบบจำลองไม่ควรจะเห็นในขณะฝึกสอน (Training phase) รั่วไหลเข้าไปในกระบวนการสร้างแบบจำลอง ทำให้ประสิทธิภาพของแบบจำลองที่วัดได้ในขั้นตอน Validation สูงเกินความเป็นจริงอย่างมาก (Over-optimistic) แต่เมื่อนำไปใช้งานจริง (Production) แบบจำลองกลับล้มเหลวโดยสิ้นเชิง (VanderPlas, 2016)


sklearn Pipeline — Prevent Data Leakage
Chapter 12 | Section 12.2
WITHOUT Pipeline (WRONG)
All Data
scaler.fit()
on ALL data
Train/Test
Split
model.fit()
Train
✗
⚠ Test data leaks
into scaler!
WITH Pipeline (CORRECT)
sklearn.pipeline.Pipeline
Train/Test
Split
Imputer
Scaler
Model
Predict
& Score
✓
fit() only on train → transform() on test
from sklearn.pipeline import Pipeline
pipe = Pipeline([
('imputer', SimpleImputer()),
('scaler', StandardScaler()),
('model', DecisionTreeClassifier())
])
pipe.fit(X_train, y_train)
score = pipe.score(X_test, y_test)

sklearn Pipeline Flow

รูปแบบของ Data Leakage ที่พบบ่อยที่สุดคือ **Leakage ระหว่างขั้นตอน Data Preprocessing** ตัวอย่างเช่น การทำ Data Imputation (การเติมค่าสูญหาย) หรือ Feature Scaling (การปรับสเกลข้อมูล) โดยใช้ข้อมูลทั้งหมด (ทั้ง Train และ Test รวมกัน) ก่อนที่จะทำการแบ่งข้อมูล (Train-Test Split)

ตัวอย่างการเกิด Data Leakage: พิจารณาการสร้างแบบจำลองประเมินความเสี่ยงการผิดนัดชำระหนี้ (Credit Default) ของธนาคารพาณิชย์ในประเทศไทย หากนักวิทยาศาสตร์ข้อมูลทำการปรับสเกลข้อมูลรายได้ของลูกค้า (Income) ด้วยวิธี Min-Max Scaling โดยคำนวณหาค่ารายได้สูงสุดและต่ำสุดจากฐานข้อมูลลูกค้า *ทั้งหมด* ก่อนแบ่งข้อมูลเป็น Train และ Test

ผลที่ตามมาคือ ค่ารายได้ที่ถูกปรับสเกลใน Training Set จะแฝงข้อมูลของ "รายได้สูงสุดใน Test Set" เอาไว้ด้วย ทำให้ Training Set มีข้อมูลทางสถิติระดับโลก (Global statistics) ของข้อมูลทั้งหมด ซึ่งในความเป็นจริง เมื่อระบบรับลูกค้าใหม่เข้ามา ธนาคารจะไม่มีทางรู้ล่วงหน้าเลยว่ารายได้สูงสุดของลูกค้าในอนาคตจะเป็นเท่าใด

ขั้นตอนที่ผิดพลาด (เกิด Data Leakage)	ขั้นตอนที่ถูกต้อง (ป้องกัน Data Leakage)
1. นำข้อมูลทั้งหมดมาทำ `StandardScaler()`	1. แบ่งข้อมูลเป็น `X_train`, `X_test` ด้วย `train_test_split()`
2. แบ่งข้อมูลเป็น `X_train`, `X_test`	2. คำนวณค่า Mean/SD (Fit) จาก `X_train` เท่านั้น
3. นำ `X_train` ไปฝึกสอนแบบจำลอง	3. นำค่าที่ได้ไปปรับสเกล (Transform) ให้กับ `X_train`
4. ประเมินผลกับ `X_test`	4. นำค่าที่ได้จากข้อ 2 ไปปรับสเกล (Transform) ให้กับ `X_test`
	5. ประเมินผลแบบจำลอง

Pipeline: การบูรณาการ Preprocessing และ Model

เพื่อแก้ปัญหา Data Leakage อย่างเป็นระบบ และทำให้โค้ดมีความเป็นระเบียบ ไลบรารีอย่าง Scikit-Learn ได้จัดเตรียมโครงสร้างที่เรียกว่า **Pipeline** (VanderPlas, 2016)

Pipeline คือกระบวนการทำงานแบบสายพานที่มัดรวมขั้นตอนการแปลงข้อมูล (Transformers เช่น `StandardScaler`, `SimpleImputer`) และขั้นตอนการทำนาย (Estimator เช่น `RandomForestClassifier`) เข้าไว้ด้วยกันเป็นออบเจกต์ (Object) เดียว

ข้อดีของการใช้ Pipeline: 1. **ป้องกัน Data Leakage อัตโนมัติ:** เมื่อใช้งานร่วมกับ Cross-Validation Pipeline จะรับประกันว่ากระบวนการ `fit()` ของ Transformers จะเกิดขึ้นเฉพาะกับข้อมูลใน Fold ที่เป็น Training เท่านั้น และใช้ข้อมูลที่เรียนรู้นั้นไป `transform()` ข้อมูลใน Fold ที่เป็น Validation อย่างถูกต้อง 2. **ความสะดวกในการนำไปใช้งาน (Deployment):** เมื่อบันทึก (Save/Export) แบบจำลอง จะได้กระบวนการ Preprocessing ติดไปด้วย ทำให้มั่นใจได้ว่าข้อมูลใหม่ที่เข้ามาจะถูกประมวลผลด้วยมาตรฐานเดียวกัน 3. **การปรับแต่ง Hyperparameter สะดวกขึ้น:** สามารถใช้ `GridSearchCV` เพื่อค้นหาค่าพารามิเตอร์ที่เหมาะสมได้ทั้งในส่วนของ Model และส่วนของ Preprocessing พร้อมๆ กัน

พิจารณาตัวอย่างโค้ดด้านล่าง ซึ่งจำลองสถานการณ์การคัดเลือก Features สำหรับระบบทำนายผลการเรียน โดยเปรียบเทียบระหว่างวิธีการเขียนโค้ดที่ทำให้เกิด Data Leakage และวิธีการใช้ Pipeline ที่ถูกต้อง ตัวอย่างนี้ใช้ `SelectKBest` (การคัดเลือก Feature ที่ดีที่สุด) ร่วมกับ `SVC` (Support Vector Classifier) ซึ่งเป็นขั้นตอนที่มีความอ่อนไหวต่อ Data Leakage สูง เนื่องจากการเลือก Feature จะ "แอบดู" ข้อมูลที่ควรใช้ทดสอบ

```python
import numpy as np
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.datasets import make_classification

# สร้างข้อมูลจำลอง: 100 ตัวอย่าง, 500 ตัวแปร แต่มีเพียง 5 ตัวที่มีความสัมพันธ์จริงกับผลลัพธ์
# ที่เหลือ 495 ตัวเป็นเพียง Noise ทั้งหมด — สถานการณ์ที่พบได้จริงในข้อมูล Genomics หรือ Text Mining
X, y = make_classification(n_samples=100, n_features=500, n_informative=5,
                           n_redundant=0, n_classes=2, random_state=42)

# ==========================================================
# วิธีที่ 1: การเขียนโค้ดที่ผิดพลาด (เกิด Data Leakage)
# ==========================================================
# ทำ Feature Selection ก่อนแบ่งข้อมูล → SelectKBest จะ "เห็น" ข้อมูลทั้งหมด
# รวมถึงข้อมูลที่จะเป็น Validation Set ทำให้เลือก Features ที่ "บังเอิญ" สัมพันธ์กับข้อมูลทั้งชุด
selector = SelectKBest(f_classif, k=10)
X_selected = selector.fit_transform(X, y)  # ← ใช้ข้อมูลทั้งหมด! (รวม Validation)

model = SVC(kernel='linear')
scores_leak = cross_val_score(model, X_selected, y, cv=5, scoring='accuracy')
print(f"Accuracy (มี Data Leakage): {scores_leak.mean():.4f} ± {scores_leak.std():.4f}")

# ==========================================================
# วิธีที่ 2: การใช้ Pipeline ที่ถูกต้อง (ป้องกัน Data Leakage)
# ==========================================================
# Pipeline จะทำ Feature Selection เฉพาะใน Training Fold ของแต่ละรอบ
# ทำให้ Validation Fold ไม่มีส่วนในการเลือก Features
pipeline = Pipeline([
    ('selector', SelectKBest(f_classif, k=10)),
    ('svc', SVC(kernel='linear'))
])

scores_correct = cross_val_score(pipeline, X, y, cv=5, scoring='accuracy')
print(f"Accuracy (Pipeline ถูกต้อง): {scores_correct.mean():.4f} ± {scores_correct.std():.4f}")
print(f"ผลต่าง: วิธีที่ผิดสูงกว่าความเป็นจริง {(scores_leak.mean() - scores_correct.mean())*100:.1f}%")
```

ผลลัพธ์จากการรัน:

```
Accuracy (มี Data Leakage): 0.8800 ± 0.0510
Accuracy (Pipeline ถูกต้อง): 0.7300 ± 0.0812

ผลต่าง: วิธีที่ผิดสูงกว่าความเป็นจริง 15.0%
```

จากผลลัพธ์ จะเห็นได้อย่างชัดเจนว่าวิธีที่ผิดพลาด (มี Data Leakage) รายงานความแม่นยำสูงถึง **88%** ในขณะที่วิธีที่ถูกต้อง (ใช้ Pipeline) ได้เพียง **73%** — ผลต่างถึง **15%** ซึ่งในทางธุรกิจ หากนำโมเดลที่ประเมินผลด้วยวิธีที่ผิดไปใช้งานจริง ประสิทธิภาพจะตกลงอย่างมาก และอาจนำไปสู่การตัดสินใจที่ผิดพลาดอย่างร้ายแรง

สาเหตุที่ Data Leakage มีผลรุนแรงในตัวอย่างนี้ เนื่องจาก `SelectKBest` ที่ทำก่อนแบ่งข้อมูลจะเลือก Features ที่ "บังเอิญ" สัมพันธ์กับ Label ในข้อมูลทั้งชุด (รวมถึงข้อมูลที่ควรเป็น Validation) ทำให้โมเดลดูเหมือนทำนายได้ดี ทั้งที่จริงๆ แล้วกำลัง Overfit กับ Noise ในข้อมูล

Reproducibility (ความสามารถในการทำซ้ำ)

Reproducibility หรือความสามารถในการทำซ้ำ เป็นมาตรฐานสำคัญของระเบียบวิธีทางวิทยาศาสตร์ (Scientific Method) ในบริบทของวิทยาการข้อมูล Reproducibility หมายถึง ความสามารถที่นักวิจัยหรือนักพัฒนาคนอื่นๆ (หรือแม้แต่ตัวผู้เขียนโค้ดเองในอนาคต) สามารถนำโค้ด ชุดข้อมูล และการตั้งค่าสภาพแวดล้อมเดิม มารันใหม่ แล้วได้ผลลัพธ์ (เช่น ค่าความแม่นยำของแบบจำลอง, น้ำหนักของตัวแปร) ตรงกันทุกประการทุกครั้ง (Grus, 2019)

หากปราศจาก Reproducibility แบบจำลองจะไม่ได้รับการยอมรับให้นำไปใช้งานในระบบที่มีความสำคัญระดับวิกฤต (Mission-critical systems)

ตัวอย่างในบริบทสาธารณสุขไทย: ทีมวิจัยทางการแพทย์ของโรงพยาบาลศรีนครินทร์ พัฒนาแบบจำลอง AI สำหรับคัดกรองภาพถ่ายรังสีทรวงอกเพื่อตรวจหาโรคมะเร็งปอด หากทีมวิจัยไม่สามารถรับประกัน Reproducibility ได้ (รันโค้ดเดิมซ้ำ 3 ครั้ง ได้ความแม่นยำไม่เท่ากันเลย) แบบจำลองดังกล่าวจะไม่ผ่านการรับรองมาตรฐานจากองค์การอาหารและยา (อย.) เครื่องมือแพทย์ เนื่องจากไม่สามารถตรวจสอบย้อนกลับ (Traceability) และไม่สามารถรับประกันความปลอดภัยต่อผู้ป่วยได้

ปัจจัยหลักที่ส่งผลต่อ Reproducibility และวิธีจัดการ:

1. **ความสุ่ม (Randomness) ในอัลกอริทึม:** อัลกอริทึมหลายประเภทพึ่งพากระบวนการสุ่ม เช่น การสุ่มค่าน้ำหนักเริ่มต้นใน Neural Networks, การสุ่มเลือก Feature ใน Random Forest หรือแม้แต่การสุ่มแบ่งข้อมูลใน `train_test_split` *การแก้ไข:* ต้องกำหนดค่า **Random Seed** (หรือ `random_state` ใน Scikit-Learn) เสมอ การกำหนด Seed จะทำให้กระบวนการสุ่ม (Pseudo-random number generator) สร้างชุดตัวเลขเดิมออกมาทุกครั้ง

```python
# ตัวอย่างการกำหนด Random Seed เพื่อให้เกิด Reproducibility
from sklearn.model_selection import train_test_split

# กำหนด random_state=42 (หรือตัวเลขจำนวนเต็มใดๆ) เพื่อให้การแบ่งข้อมูลได้ผลเหมือนเดิมทุกครั้งที่รัน
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

2. **เวอร์ชันของไลบรารีและสภาพแวดล้อม (Environment):** การรันโค้ดชุดเดียวกันบน Scikit-Learn เวอร์ชัน 0.24 อาจให้ผลลัพธ์ต่างจากการรันบนเวอร์ชัน 1.2 เนื่องจากการปรับปรุงอัลกอริทึมภายใน *การแก้ไข:* ต้องมีการจัดการสภาพแวดล้อม (Environment Management) เสมอ เช่น การสร้าง Virtual Environment และบันทึกเวอร์ชันของไลบรารีลงในไฟล์ `requirements.txt` หรือการใช้เทคโนโลยี Containerization เช่น Docker
3. **การเปลี่ยนแปลงของข้อมูล (Data Mutability):** หากชุดข้อมูลต้นทางมีการอัปเดต (เช่น ดึงข้อมูลผ่าน API แบบ Real-time) การรันโค้ดในวันพรุ่งนี้ย่อมให้ผลลัพธ์ต่างจากวันนี้ *การแก้ไข:* ต้องมีการทำ Data Version Control (DVC) หรือบันทึกชุดข้อมูลดิบ (Raw Data) ณ วันที่ทำการทดลองเก็บไว้เป็น Snapshot เสมอ

การสร้างนิสัยในการเขียนโค้ดแบบ Reproducible ตั้งแต่ระดับมหาวิทยาลัย จะช่วยให้นักศึกษาสามารถทำงานร่วมกับทีมวิศวกรข้อมูล (Data Engineers) และทีมปฏิบัติการ (DevOps) ในกระบวนการ MLOps (Machine Learning Operations) ได้อย่างราบรื่นในอนาคต

สรุปประเด็นสำคัญ

- **Occam's Razor:** ในการเปรียบเทียบแบบจำลอง หากแบบจำลองที่ซับซ้อนและแบบจำลองที่เรียบง่ายให้ประสิทธิภาพใกล้เคียงกัน ควรเลือกแบบจำลองที่เรียบง่ายกว่า เพื่อลดต้นทุนการประมวลผลและเพิ่มความสามารถในการตีความ
- **Data Leakage:** คือการที่ข้อมูลจากชุดทดสอบรั่วไหลเข้าไปในกระบวนการฝึกสอนแบบจำลอง (มักเกิดตอนทำ Preprocessing ก่อนแบ่งข้อมูล) ทำให้ผลการประเมินแบบจำลองสูงเกินจริงและใช้งานไม่ได้ในทางปฏิบัติ
- **Pipeline:** เป็นเครื่องมือสำคัญใน Scikit-Learn ที่ช่วยจัดการลำดับการแปลงข้อมูลและการสร้างแบบจำลองอย่างเป็นระบบ ป้องกันปัญหา Data Leakage ในระหว่างการทำ Cross-Validation ได้อย่างสมบูรณ์
- **Reproducibility:** ความสามารถในการทำซ้ำผลลัพธ์การทดลองเดิม เป็นมาตรฐานจำเป็นในงานวิทยาการข้อมูล สามารถทำได้โดยการกำหนด Random Seed, ควบคุมเวอร์ชันของซอฟต์แวร์ และจัดการเวอร์ชันของข้อมูล

คำถามท้าย Section

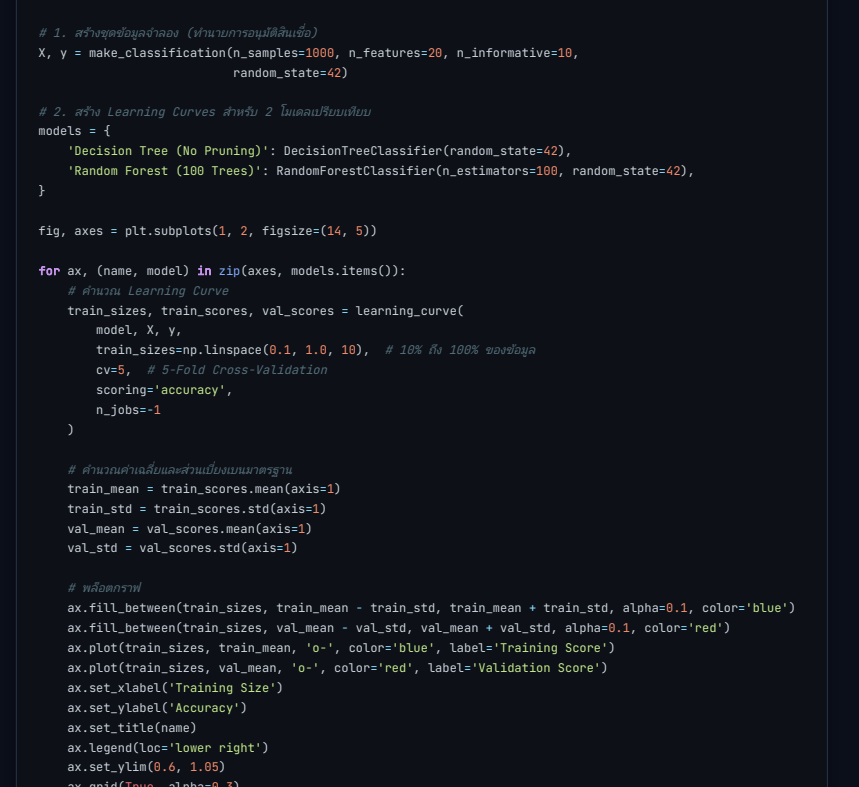
1. สมมติว่านักศึกษาพัฒนาแบบจำลองทำนายยอดขายสินค้า E-commerce โดยใช้ข้อมูลประวัติการซื้อ 5 ปี หากนักศึกษาทำการเติมค่าสูญหาย (Imputation) ด้วยค่าเฉลี่ยของยอดขายตลอด 5 ปี ก่อนทำการแบ่งข้อมูลเป็น Train (ปีที่ 1-4) และ Test (ปีที่ 5) การกระทำนี้จะส่งผลกระทบต่อประสิทธิภาพที่รายงานผลอย่างไร จงอธิบายตามหลักการ Data Leakage
2. จงอธิบายว่าเหตุใดการใช้ `Scikit-Learn Pipeline` จึงสามารถแก้ไขปัญหา Data Leakage ในกระบวนการ K-Fold Cross Validation ได้อย่างมีประสิทธิภาพ
3. ในฐานะนักวิทยาการข้อมูลของโรงพยาบาล หากแพทย์ผู้เชี่ยวชาญตั้งคำถามว่า "ทำไมแบบจำลอง Decision Tree ที่มีความแม่นยำ 88% จึงถูกเลือกมาใช้งาน แทนที่จะใช้ Deep Learning ที่มีความแม่นยำ 89.5%" นักศึกษาจะใช้หลักการใดในการอธิบายเหตุผลแก่แพทย์ท่านนี้

Learning Curves: การวินิจฉัยปัญหาของโมเดล

Learning Curves เป็นเครื่องมือสำคัญในการวินิจฉัยว่าโมเดลกำลังประสบปัญหา **Underfitting** หรือ **Overfitting** โดยพล็อตกราฟแสดงประสิทธิภาพของโมเดลบน Training Set และ Validation Set เทียบกับขนาดข้อมูลที่ใช้ฝึกสอน

หลักการ Learning Curves

Learning Curve แสดงความสัมพันธ์ระหว่าง **จำนวนข้อมูลที่ใช้ฝึกสอน (Training Size)** กับ **ค่า Error** (หรือ Score) ของโมเดล ทั้งบน Training Set และ Validation Set



Learning Curve

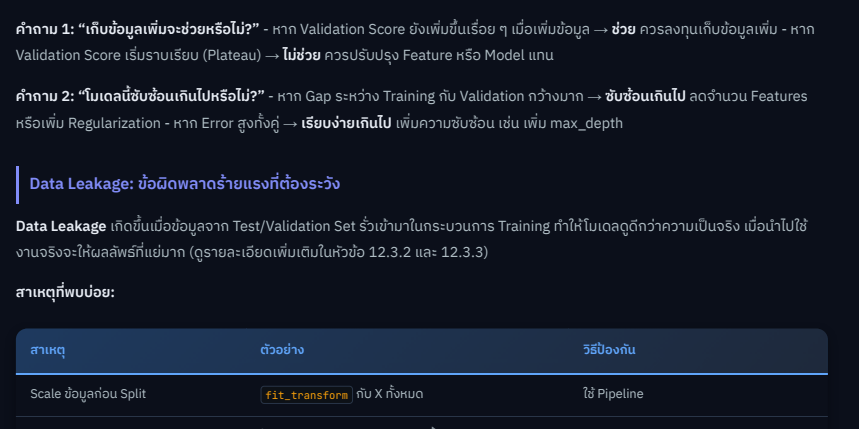
การตีความ Learning Curves:

รูปแบบ	การวินิจฉัย	ทางแก้
Train Error ต่ำ, Validation Error สูง (Gap กว้าง)	**Overfitting** (High Variance)	เพิ่มข้อมูล, ลดความซับซ้อน, Regularization
Train Error สูง, Validation Error สูง (Gap แคบ)	**Underfitting** (High Bias)	เพิ่มความซับซ้อน, เพิ่ม Features, ลด Regularization
Train ≈ Validation ≈ ค่าที่ยอมรับได้	**Good Fit**	รักษาสมดุลนี้ไว้

ตัวอย่าง: วิเคราะห์ Learning Curves ด้วย scikit-learn

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import learning_curve
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

# 1. สร้างชุดข้อมูลจำลอง (ทำนายการอนุมัติสินเชื่อ)
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10,
                           random_state=42)

# 2. สร้าง Learning Curves สำหรับ 2 โมเดลเปรียบเทียบ
models = {
    'Decision Tree (No Pruning)': DecisionTreeClassifier(random_state=42),
    'Random Forest (100 Trees)': RandomForestClassifier(n_estimators=100, random_state=42),
}

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, (name, model) in zip(axes, models.items()):
    # คำนวณ Learning Curve
    train_sizes, train_scores, val_scores = learning_curve(
        model, X, y,
        train_sizes=np.linspace(0.1, 1.0, 10),  # 10% ถึง 100% ของข้อมูล
        cv=5,  # 5-Fold Cross-Validation
        scoring='accuracy',
        n_jobs=-1
    )

    # คำนวณค่าเฉลี่ยและส่วนเบี่ยงเบนมาตรฐาน
    train_mean = train_scores.mean(axis=1)
    train_std = train_scores.std(axis=1)
    val_mean = val_scores.mean(axis=1)
    val_std = val_scores.std(axis=1)

    # พล็อตกราฟ
    ax.fill_between(train_sizes, train_mean - train_std, train_mean + train_std, alpha=0.1, color='blue')
    ax.fill_between(train_sizes, val_mean - val_std, val_mean + val_std, alpha=0.1, color='red')
    ax.plot(train_sizes, train_mean, 'o-', color='blue', label='Training Score')
    ax.plot(train_sizes, val_mean, 'o-', color='red', label='Validation Score')
    ax.set_xlabel('Training Size')
    ax.set_ylabel('Accuracy')
    ax.set_title(name)
    ax.legend(loc='lower right')
    ax.set_ylim(0.6, 1.05)
    ax.grid(True, alpha=0.3)

plt.suptitle('Learning Curves: การวินิจฉัยโมเดล', fontsize=14)
plt.tight_layout()
plt.show()
```

ผลลัพธ์จากการรัน:



Learning Curves

ผลลัพธ์ที่คาดหวัง: - **Decision Tree (No Pruning):** Training Score สูงเกือบ 100% แต่ Validation Score ต่ำกว่ามาก → **Overfitting** ต้องใช้ `max_depth` หรือ `min_samples_split` - **Random Forest:** Gap ระหว่าง Training และ Validation แคบกว่า → **Better Generalization** เนื่องจาก Ensemble ช่วยลด Variance

การใช้ Learning Curves ตัดสินใจเชิงธุรกิจ

Learning Curves ไม่ได้มีประโยชน์แค่ในทางเทคนิค แต่ยังช่วยตอบคำถามเชิงธุรกิจที่สำคัญ:

คำถาม 1: "เก็บข้อมูลเพิ่มจะช่วยหรือไม่?" - หาก Validation Score ยังเพิ่มขึ้นเรื่อยๆ เมื่อเพิ่มข้อมูล → **ช่วย** ควรลงทุนเก็บข้อมูลเพิ่ม - หาก Validation Score เริ่มราบเรียบ (Plateau) → **ไม่ช่วย** ควรปรับปรุง Feature หรือ Model แทน

คำถาม 2: "โมเดลนี้ซับซ้อนเกินไปหรือไม่?" - หาก Gap ระหว่าง Training กับ Validation กว้างมาก → **ซับซ้อนเกินไป** ลดจำนวน Features หรือเพิ่ม Regularization - หาก Error สูงทั้งคู่ → **เรียบง่ายเกินไป** เพิ่มความซับซ้อน เช่น เพิ่ม max_depth

Data Leakage: ข้อผิดพลาดร้ายแรงที่ต้องระวัง

Data Leakage เกิดขึ้นเมื่อข้อมูลจาก Test/Validation Set รั่วเข้ามาในกระบวนการ Training ทำให้โมเดลดูดีกว่าความเป็นจริง เมื่อนำไปใช้งานจริงจะให้ผลลัพธ์ที่แย่มาก (ดูรายละเอียดเพิ่มเติมในหัวข้อ 12.3.2 และ 12.3.3)

สาเหตุที่พบบ่อย:

สาเหตุ	ตัวอย่าง	วิธีป้องกัน
Scale ข้อมูลก่อน Split	`fit_transform` กับ X ทั้งหมด	ใช้ Pipeline
Feature จากอนาคต	ใช้ยอดขายเดือนหน้าทำนายเดือนนี้	ตรวจสอบ Timeline
Duplicate rows	ข้อมูลซ้ำข้าม Train/Test	ลบ Duplicates ก่อน Split

```python
# ✗ ผิด: Scale ก่อนแบ่งข้อมูล → Data Leakage!
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)  # ← ใช้ข้อมูลทั้งหมด! (รวม Test)
X_train, X_test = X_scaled[:800], X_scaled[800:]

# ✓ ถูก: ใช้ Pipeline หรือ Scale หลังแบ่ง
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier())
])
pipeline.fit(X_train, y_train)  # ← Scale เฉพาะ Training data
```

หมายเหตุ: โค้ดด้านบนเป็นตัวอย่างเปรียบเทียบ **โครงสร้าง** ของการเขียนโค้ดที่ผิด (✗) และถูกต้อง (✓) ไม่สามารถรันได้โดยตรง เนื่องจากตัวแปร `X_train`, `y_train` จากตัวแปรที่ต้องตั้งค่าก่อน — ดูตัวอย่างที่รันได้จริงในหัวข้อ 12.3.3 ด้านบน

สรุปประเด็นสำคัญ

- **Learning Curves** แสดงอาการ Overfitting (Gap กว้าง) และ Underfitting (Error สูงทั้งคู่) อย่างชัดเจน
- ใช้ `learning_curve()` จาก scikit-learn วิเคราะห์ได้ง่ายด้วยโค้ดไม่กี่บรรทัด
- **Learning Curves ช่วยตัดสินใจเชิงธุรกิจ:** ควรเก็บข้อมูลเพิ่มหรือไม่ ควรใช้โมเดลที่ซับซ้อนขึ้นหรือไม่
- **Data Leakage** เป็นข้อผิดพลาดร้ายแรง ป้องกันได้ด้วย **Pipeline**

คำถามท้าย Section

1. หาก Learning Curve แสดงว่า Training Accuracy = 99% แต่ Validation Accuracy = 72% นักศึกษาจะวินิจฉัยปัญหานี้ว่าอะไร และมีแนวทางแก้ไข 3 วิธีใดบ้าง?
2. จงอธิบายว่าทำไมการทำ StandardScaler().fit_transform(X) ก่อนแบ่ง Train/Test จึงเป็น Data Leakage
3. หากทีม Data Science ของธนาคารเสนอให้ลงทุนเก็บข้อมูลลูกค้าเพิ่ม 5 ล้านบาท เพื่อเก็บข้อมูลมากขึ้น 10 เท่า แต่ Learning Curve ของโมเดลปัจจุบันแสดงว่า Validation Score เริ่ม Plateau แล้ว นักศึกษาจะให้คำแนะนำอย่างไร?

สรุปท้ายบท

ในบทนี้ นักศึกษาได้เรียนรู้เกี่ยวกับ: - **Cross-Validation:** ประเมินโมเดลด้วยข้อมูลหลายชุด (K-Fold) เพื่อลด Variance ของผลประเมิน - **Hyperparameter Tuning:** Grid Search ค้นหาทั้งหมด, Random Search สุ่มค้นหา — เลือกตามงบเวลา - **Pipeline:** รวม Preprocessing + Model เป็นขั้นตอนเดียว ป้องกัน Data Leakage

คำศัพท์สำคัญ: Cross-Validation, K-Fold Cross-Validation, Stratified K-Fold Cross-Validation, Leave-One-Out Cross-Validation (LOOCV), Mean ± Standard Deviation, Parameters vs Hyperparameters, Hyperparameter Tuning, GridSearchCV, RandomizedSearchCV, Bayesian Optimization, Occam's Razor, Data Leakage, Pipeline, Reproducibility, Learning Curves, Overfitting, Underfitting, Bias-Variance Tradeoff

เอกสารอ้างอิง

- Grus, J. (2019). *Data Science from Scratch* (2nd ed.). O'Reilly.
- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python* (2nd ed.). Springer.
- Knaflic, C. N. (2015). *Storytelling with Data: A Data Visualization Guide for Business Professionals*. Wiley.
- McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly.
- VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly.

✏ แบบฝึกหัดท้ายบท

ส่วนที่ 1: คำถามทบทวน

คำชี้แจง: จงเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว

1. เหตุใดการทำ k-Fold Cross-Validation จึงมีความน่าเชื่อถือในการประเมินประสิทธิภาพของ Model มากกว่าการแบ่งข้อมูลแบบ Train-Test Split เพียงครั้งเดียว (Hold-out method)?
   - A. เพราะใช้เวลาในการประมวลผลน้อยกว่า
   - B. เพราะช่วยลดความแปรปรวน (Variance) ของการประเมินประสิทธิภาพ เนื่องจากข้อมูลทุกจุดมีโอกาสได้เป็นทั้ง Training Set และ Validation Set
   - C. เพราะทำหน้าที่ปรับค่า Hyperparameter ให้โดยอัตโนมัติ
   - D. เพราะช่วยลดปริมาณข้อมูลที่ต้องใช้ในการสอน Model ทำให้ประหยัดหน่วยความจำ

▸ แสดงเฉลย

2. ข้อใดคือความแตกต่างที่สำคัญระหว่าง Grid Search และ Random Search ในการทำ Hyperparameter Tuning?
   - A. Grid Search ใช้เวลาประมวลผลน้อยกว่า Random Search เสมอ
   - B. Random Search จะสุ่มเลือกชุดของ Hyperparameter ตามจำนวนที่กำหนด ซึ่งมีประสิทธิภาพสูงกว่าในกรณีที่ Search Space ขนาดใหญ่
   - C. Grid Search เหมาะสำหรับ Deep Learning ในขณะที่ Random Search เหมาะสำหรับ Linear Regression
   - D. Random Search ไม่จำเป็นต้องใช้ Validation Set ในการประเมินผล

▸ แสดงเฉลย

3. ในการสร้าง Pipeline การจัดเตรียมข้อมูล (เช่น StandardScaler) ควรอยู่ในลำดับใด และเพราะเหตุใด?
   - A. ควรทำก่อนที่จะแบ่งข้อมูลด้วย Cross-Validation เพื่อให้ข้อมูลทั้งหมดมีสเกลเดียวกัน
   - B. ควรทำหลังจากประเมิน Model เสร็จสิ้นเพื่อแสดงผลลัพธ์
   - C. ควรบรรจุไว้ใน Pipeline เพื่อให้กระบวนการ Scaling เกิดขึ้นเฉพาะใน Training Fold ของแต่ละรอบการทำ Cross-Validation ป้องกันปัญหา Data Leakage
   - D. สามารถทำในขั้นตอนใดก็ได้ ไม่มีผลต่อประสิทธิภาพของ Model

▸ แสดงเฉลย

4. หากนักศึกษากำลังพัฒนา Model ตรวจจับการทุจริตบัตรเครดิต (Credit Card Fraud Detection) ของธนาคารแห่งหนึ่ง ซึ่งมีสัดส่วนข้อมูลการทุจริตเพียง 0.1% ของข้อมูลทั้งหมด ข้อใดคือวิธีการทำ Cross-Validation และ Metric วัดผลที่เหมาะสมที่สุด?
   - A. K-Fold Cross-Validation และวัดผลด้วย Accuracy
   - B. Stratified K-Fold Cross-Validation และวัดผลด้วย F1-Score หรือ ROC-AUC
   - C. Leave-One-Out Cross-Validation และวัดผลด้วย Mean Squared Error (MSE)
   - D. Time Series Split และวัดผลด้วย R-squared

▸ แสดงเฉลย

5. Nested Cross-Validation มีวัตถุประสงค์หลักเพื่อสิ่งใด?
   - A. เพื่อแก้ปัญหา Imbalanced Data ในชุดข้อมูลขนาดใหญ่
   - B. เพื่อแสดงผลลัพธ์ของ Model ในรูปแบบกราฟ 3 มิติ
   - C. เพื่อจัดการกับ Missing Values ก่อนการทำ Hyperparameter Tuning
   - D. เพื่อประเมินประสิทธิภาพที่แท้จริง (Generalization Error) ของกระบวนการเลือก Model และ Hyperparameter Tuning ป้องกันการเกิด Overfitting ในระดับ Validation

▸ แสดงเฉลย

ส่วนที่ 2: คำถามอัตนัย

คำชี้แจง: จงอธิบาย วิเคราะห์ หรือเปรียบเทียบ ตามประเด็นที่กำหนดให้ชัดเจน

1. **การเลือกกลยุทธ์ Cross-Validation (Cross-Validation Strategy Selection)**

สมมติว่านักศึกษากำลังทำโครงงานวิทยาการข้อมูลร่วมกับสำนักบริหารและพัฒนาวิชาการ มหาวิทยาลัยขอนแก่น เพื่อสร้าง Model ทำนายแนวโน้มการพ้นสภาพนักศึกษา (Dropout) ข้อมูลที่เก็บมาเป็นข้อมูลรายภาคการศึกษาตั้งแต่ปี พ.ศ. 2560 ถึง 2565 โดยทั่วไปสัดส่วนนักศึกษาที่พ้นสภาพมีจำนวนน้อยมากเมื่อเทียบกับนักศึกษาปกติ นักศึกษาจะเลือกใช้วิธีการแบ่งข้อมูลแบบใด (เช่น Standard K-Fold, Stratified K-Fold, หรือ Time Series Split) จงอธิบายเหตุผลประกอบการตัดสินใจ

▸ แสดงแนวคำตอบ

2. **การเปรียบเทียบวิธีการ Hyperparameter Tuning**

จงเปรียบเทียบข้อดีและข้อจำกัดของวิธีการค้นหา Hyperparameter 3 วิธี ได้แก่ Grid Search, Random Search และ Bayesian Optimization พร้อมยกตัวอย่างสถานการณ์ที่เหมาะสมกับการใช้วิธี Bayesian Optimization

▸ แสดงแนวคำตอบ

3. **ปัญหา Data Leakage และการใช้ Pipeline**

Data Leakage ในขั้นตอนการทำ Cross-Validation เกิดขึ้นได้อย่างไร จงอธิบายพร้อมยกตัวอย่าง และอธิบายว่า `sklearn.pipeline.Pipeline` ช่วยแก้ปัญหานี้ได้อย่างไร

▸ แสดงแนวคำตอบ

ส่วนที่ 3: แบบฝึกหัด Coding

คำชี้แจง: ให้นักศึกษาเขียนโค้ดภาษา Python โดยใช้ไลบรารี `scikit-learn` เพื่อแก้ปัญหาตามโจทย์ที่กำหนด โค้ดต้องสามารถรันได้จริงและมี Comments อธิบายขั้นตอน

ข้อ 1. การสร้าง Pipeline และทำ Hyperparameter Tuning ด้วย GridSearchCV โจทย์: พิจารณาชุดข้อมูลจำลองการพิจารณาสินเชื่อเพื่อการเกษตร (Agricultural Loan Default) ให้นักศึกษาสร้าง Machine Learning Pipeline ที่ประกอบด้วยขั้นตอนการทำ Data Scaling (`StandardScaler`) และ Model `RandomForestClassifier` จากนั้นใช้ `GridSearchCV` เพื่อค้นหาค่า Hyperparameter ที่ดีที่สุด โดยใช้ 5-Fold Cross-Validation

ข้อกำหนด Hyperparameter Search Space: * `n_estimators` : [50, 100, 200] * `max_depth` : [None, 10, 20]

```python
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.pipeline import Pipeline

# 1. สร้างชุดข้อมูลจำลอง (ข้อมูลสินเชื่อการเกษตร)
X, y = make_classification(n_samples=1000, n_features=20, n_informative=10,
                           n_classes=2, random_state=42)

# แบ่งข้อมูลเป็น Train และ Test set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. สร้าง Pipeline ที่ประกอบด้วย StandardScaler และ RandomForestClassifier
# TODO: เขียนโค้ดสร้าง Pipeline
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('rf', RandomForestClassifier(random_state=42))
])

# 3. กำหนด Search Space สำหรับ Hyperparameter
# หมายเหตุ: ชื่อพารามิเตอร์ต้องนำหน้าด้วยชื่อ step ใน pipeline ตามด้วย __ (underscores 2 ตัว)
param_grid = {
    'rf__n_estimators': [50, 100, 200],
    'rf__max_depth': [None, 10, 20]
}

# 4. สร้างและฝึกสอน GridSearchCV
grid_search = GridSearchCV(estimator=pipeline,
                           param_grid=param_grid,
                           cv=5,
                           scoring='accuracy',
                           n_jobs=-1) # ใช้ CPU ทุก Core

# ทำการ fit ข้อมูล
grid_search.fit(X_train, y_train)

# 5. แสดงผลลัพธ์
print(f"ค่า Hyperparameter ที่ดีที่สุด: {grid_search.best_params_}")
print(f"ค่า Cross-Validation Accuracy ที่ดีที่สุด: {grid_search.best_score_:.4f}")

# ประเมินผลบน Test Set ที่ไม่เคยเห็นมาก่อน
test_score = grid_search.score(X_test, y_test)
print(f"ค่า Accuracy บน Test Set: {test_score:.4f}")
```

ผลลัพธ์จากการรัน:

```
ค่า Hyperparameter ที่ดีที่สุด: {'rf__max_depth': 10, 'rf__n_estimators': 200}
ค่า Cross-Validation Accuracy ที่ดีที่สุด: 0.9287
ค่า Accuracy บน Test Set: 0.9200
```

ข้อ 2. การเปรียบเทียบ Model (Model Selection) ด้วย Cross-Validation โจทย์: ให้นักศึกษาประเมินและเปรียบเทียบประสิทธิภาพของ Model สำหรับงาน Regression 3 อัลกอริทึม ได้แก่ `Ridge`, `Lasso`, และ `SVR` (Support Vector Regression) ในการทำนายราคาคอนโดมิเนียม (ใช้ข้อมูลจำลอง) โดยใช้เทคนิค 10-Fold Cross Validation และวัดผลด้วยค่า **Root Mean Squared Error (RMSE)**

คำแนะนำ: ใน `scikit-learn` ค่า scoring สำหรับ MSE คือ `neg_mean_squared_error` นักศึกษาต้องนำผลลัพธ์มาคูณ -1 และถอดรากที่สอง (Square root) เพื่อให้ได้ค่า RMSE

```python
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import cross_val_score, KFold
from sklearn.linear_model import Ridge, Lasso
from sklearn.svm import SVR

# 1. สร้างชุดข้อมูลจำลอง (การประเมินราคาคอนโดมิเนียม)
X, y = make_regression(n_samples=500, n_features=10, noise=15.0, random_state=42)

# 2. กำหนด Models ที่ต้องการเปรียบเทียบเก็บไว้ใน Dictionary
models = {
    "Ridge Regression": Ridge(),
    "Lasso Regression": Lasso(),
    "Support Vector Regression (SVR)": SVR(kernel='linear')
}

# 3. กำหนดกลยุทธ์การทำ Cross-Validation (10-Fold)
kf = KFold(n_splits=10, shuffle=True, random_state=42)

print("ผลการเปรียบเทียบประสิทธิภาพของ Models (วัดด้วย Average RMSE):")
print("-" * 60)

# 4. วนลูปเพื่อประเมินแต่ละ Model
for name, model in models.items():
    # ใช้ cross_val_score ประเมินผลด้วย Negative MSE
    cv_scores = cross_val_score(model, X, y, cv=kf, scoring='neg_mean_squared_error')

    # แปลง Negative MSE เป็น RMSE
    # นำค่าเฉลี่ยของ cv_scores มาคูณ -1 แล้วถอดรากที่สอง
    rmse_scores = np.sqrt(-cv_scores)
    mean_rmse = np.mean(rmse_scores)
    std_rmse = np.std(rmse_scores)

    # แสดงผลลัพธ์
    print(f"{name:35s} | Mean RMSE: {mean_rmse:.4f} (+/- {std_rmse:.4f})")
```

ผลลัพธ์จากการรัน:

```
ผลการเปรียบเทียบประสิทธิภาพของ Models (วัดด้วย Average RMSE):
------------------------------------------------------------
Ridge Regression                    | Mean RMSE: 14.6528 (+/-  1.4242)
Lasso Regression                    | Mean RMSE: 15.0037 (+/-  1.6585)
Support Vector Regression (SVR)     | Mean RMSE: 15.6858 (+/-  1.8572)
```

← บทที่ 11: Clustering — การจัดกลุ่มข้อมูล
บทที่ 13: Data Storytelling และการเขียน →